Sistema de Reservas de Citas

Programación II

Maestro: Juan Rosario

Elaborado por:

Johaly Concepcion Polanco

Índice del contenido

Introducción 2

Objetivos 3

Reglas de negocio 4

Flujo básico de la aplicación 5

Arquitectura del sistema 6

Patrones de diseño 7-12

Tecnologías utilizadas 13

Capturas y diagramas 14

Introducción

El Sistema de Reservas de Citas está diseñado para optimizar la gestión de turnos en servicios que requieren atención programada, como clínicas, salones de belleza o asesorías. Su objetivo principal es garantizar que los usuarios puedan reservar citas de forma segura, eficiente y sin conflictos de horarios**.**

Entre sus funcionalidades se destacan:

- Generación automática de horarios disponibles.
- Validación de disponibilidad para evitar duplicidad de citas.
- Notificaciones por correo electrónico para confirmar o cancelar reservas.
- Registro completo de todas las acciones realizadas dentro de la plataforma para fines de auditoría.

Objetivos

Objetivo General:

Desarrollar un sistema de reservas de citas que integre validaciones de disponibilidad, notificaciones automáticas y registro completo de actividades.

Objetivos Específicos:

- Centralizar la gestión de turnos, fechas y estaciones de atención.
- Generar automáticamente los horarios disponibles para cada sesión.
- Permitir a los usuarios reservar citas de manera segura.
- Enviar notificaciones automáticas vía correo electrónico.
- Registrar todas las acciones realizadas (reserva, modificación, cancelación) para auditoría.

Reglas del negocio

1. Cada usuario solo puede tener una cita activa por día.
2. Las reservas deben realizarse dentro de los horarios configurados.
3. Los horarios se bloquean automáticamente cuando se llenan los cupos.
4. Cada estación solo puede atender una cita por turno.
5. La duración mínima de las citas es de 5 minutos. (Extra)
6. Las estaciones pueden tener cupos personalizables según la necesidad del servicio.
7. Todas las acciones (reserva, cancelación, modificación) se registran automáticamente para auditoría.
8. Un día puede tener varios turnos programados en distintas fechas.(Extra)

Funcionalidades

Usuario

- Registrarse
- Ver los horarios disponibles
- Reservar un cupo por día
- Confirmar la reserva
- Cancelar la reserva
- Ver reservas realizadas

Admin

- Cambiar configuración (Duración de citas, rango de los turnos, cupos por estaciones)
- Agendar sesiones o cabinas de atención para 1 o varios días
- Ver los turnos habilitados
- Ver los usuarios admin registrados

Flujo básico de la aplicación

Roles y funciones

Administrador:

- Configura duración de citas y tiempo de cada turno.
- Planifica sesiones de atención y define la cantidad de estaciones por sesión.
- Resultado: el sistema valida la información y genera registros en las entidades correspondientes (Cita, Turno, Slot, Estación, etc.).

Usuario:

- Visualiza días y slots disponibles según la configuración del administrador.
- Interactúa seleccionando un slot, recibiendo correo de confirmación con enlace para confirmar o cancelar la reserva.
- El enlace de confirmación tiene un tiempo límite definido por el sistema.

Generación de slots

A partir de la configuración de turnos y duración de citas, el sistema calcula automáticamente la cantidad de slots disponibles por sesión.

Cada slot queda asociado a un turno y a una estación específica.

Validaciones:

- Evita duplicidad de reservas en el mismo slot.
- Controla que cada estación no exceda su capacidad máxima.

Confirmación y cancelación

Confirmación:

- Actualiza el estado de la reserva.
- Garantiza que la reserva no sea gestionada automáticamente por el servicio de Clean Up.

Cancelación:

- Utiliza un token único enviado por correo.
- Libera el slot correspondiente, aumentando la disponibilidad del sistema (+1).

Servicio de mantenimiento (Clean Up)

- Revisa periódicamente las reservas con fechas vencidas.
- Cancela automáticamente las reservas no confirmadas que hayan excedido el tiempo límite de confirmación.

Arquitectura del sistema

El sistema sigue la arquitectura Onion, basada en capas para separar responsabilidades y mantener código limpio y escalable.

Capa de Dominio

- Contiene las entidades principales: Usuario, Estación, Turno, DíaHabilitado, Log, Reserva, Role, Slot, TokenCancelacion, Planificacion.
- Define interfaces para repositorios y servicios siguiendo el patrón Dependency Inversion, asegurando que la infraestructura dependa de la lógica de dominio y no al revés.

Capa de Aplicación

- Gestiona casos de uso y coordina la lógica del sistema.
- Contiene DTOs para transmitir información entre capas sin exponer datos sensibles.
- Funcionalidades: validar disponibilidad, gestionar reservas y cancelaciones, coordinar notificaciones.

Capa de Infraestructura

- Gestiona recursos físicos y externos.
- Persistencia: Entity Framework, logs y archivos de configuración.
- Autenticación: JWT.
- Repositorios: operaciones CRUD por entidad, usando Repository y Unit of Work.
- Servicios externos:
- Encriptación y verificación de contraseñas (BCrypt).
- Clean Up Service para limpieza de reservas vencidas.
- SmtpEmailAdapter para envío de correos (MailKit).
- Logger con patrones Singleton y Strategy.

Capa Web

- **Client:** Frontend en React.js con componentes y páginas interactivas.
- **Server:** Backend en ASP.NET Core con controladores para autenticación, planificación, reservas y gestión de usuarios.

Patrones de diseños utilizados

- **Singleton:** Instancia única para logger global.
- **Adapter:** Conecta servicios externos (MailKit) a la interfaz interna.
- **Repository:** Operaciones CRUD para cada entidad.
- **Unit of Work:** Agrupa operaciones en la base de datos para mantener consistencia.
- **Strategy:** Permite cambiar dinámicamente la forma de registrar logs.

Explicación de uso:

- **Adapter:** integra MailKit a la lógica interna de envío de correos.

  EL mailkit Smtp tiene una interfaz diferente a la que se definió en la capa de dominio. Se utiliza para simplificar el flujo de acciones en capas como web y aplicación, de modo que la complejidad es abstraída por interfaz y es implementada en la capa de infraestructura, permitiendo no solo que se defina el servicio concreto separado de la lógica de negocio, sino que también permite intercambiar entre otros servicios de correo.

```
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Dominio.Interfaces
{
    6 referencias
    public interface IEmailService
    {
        2 referencias
        Task EnviarCorreoAsync(string destino, string asunto, string mensaje);
    }
}
```

```
public class SmtpEmailAdapter : IEmailService
{
    private readonly string _remitente = "noreply.sistemacitas@gmail.com";
    private readonly string _contraseñaApp = "maoz juwg swmk gucp\r\n";

    2 referencias
    public async Task EnviarCorreoAsync(string destino, string asunto, string mensaje)
    {
        try
        {
            var emailMessage = new MimeMessage();

            // Remitente
            emailMessage.From.Add(new MailboxAddress("Sistema de Reservas", _remitente));

            // Destinatario
            emailMessage.To.Add(MailboxAddress.Parse(destino));

            // Asunto y cuerpo
            emailMessage.Subject = asunto;
            emailMessage.Body = new TextPart("plain") { Text = mensaje };

            using var client = new SmtpClient();

            // Conexión a Gmail
            await client.ConnectAsync("smtp.gmail.com", 587, MailKit.Security.SecureSocketOptions.StartTls);

            // Autenticación con contraseña de aplicación
            await client.AuthenticateAsync(_remitente, _contraseñaApp);

            // Envío
            await client.SendAsync(emailMessage);

            await client.DisconnectAsync(true);
        }
        catch (SmtpCommandException ex)
        {
            Console.WriteLine($"Error en comando SMTP: {ex.Message}");
            throw new InvalidOperationException();
        }
        catch (SmtpProtocolException ex)
        {
            Console.WriteLine($"Error en protocolo SMTP: {ex.Message}");
            throw new InvalidOperationException();
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error general al enviar correo: {ex.Message}");
            throw new InvalidOperationException();
        }
    }
}
}
```

- **Singleton:** asegura un Logger único para toda la aplicación.

  En los controladores principalmente, se presenta la necesidad de tener el control de las excepciones, y el singleton Logger permite tener acceso simple a una función para almacenar los datos de eventos y/ errores. No es necesario inyectarlo en todos los controladores ya que al ser una sola instancia, todos pueden acceder al mismo de la misma forma y sin complicaciones.

```
catch (Exception ex)
{
    await LoggerContext.GetLogger().Registrar(new Log
    {
        FechaHora = DateTime.Now,
        Tipo = "Exception",
        Mensaje = ex.Message
    });
```

```
var logger = LoggerContext.GetLogger;
try
{
    await _servico.Actualizar(config);


    LoggerContext.GetLogger().SetDatabaseLogger();
    await LoggerContext.GetLogger().Registrar(new Log()
    {
        FechaHora = DateTime.UtcNow,
        Tipo = "Modificacion",
        Mensaje = "La configuración fue cambiada"
    });
```

- **Strategy:** alterna tipos de registro de errores y actividades.

  Si bien la funcionalidad de loggeo es bastante útil para almacenar cualquier evento, se presenta la necesidad de loggear algo en concreto, ya sea un error o evento. Para ello se hace uso de estrategy, el cual permite en el lugar donde se utiliza el Logger cambiar de opción de Loggeo.

  LoggerContext: representa la entidad que gestiona el cambio de estrategias y delega los algoritmos al tipo de estrategia. También es el singleton.

```
56 referencias
public class LoggerContext
{
    private static LoggerContext _instancia;
    private ILoggerStrategy _strategy;
    private static IRepositorio<Log> _repo;

    52 referencias
    public static LoggerContext GetLogger(IRepositorio<Log> repo = null)
    {
        if (_instancia == null)
        {
            if (repo == null) throw new ArgumentNullException(nameof(repo));
            _repo = repo;
            _instancia = new LoggerContext();
        }
        return _instancia;
    }

    1 referencia
    private LoggerContext() { }

    4 referencias
    public void SetDatabaseLogger()
    {
        _strategy = new DatabaseLogger(_repo);
    }

    18 referencias
    public void SetFileLogger()
    {
        _strategy = new FileLogger();
    }

    23 referencias
    public async Task Registrar(Log log)
    {
        if (_strategy == null) throw new InvalidOperationException("Debe establecerse una estrategia.");
        await _strategy.Registrar(log);
    }
}
```

Ejemplo:

FileLogger: almacena los logs en un archivo txt. En la aplicación es utilizado para registrar errores de ejecución

```
namespace Infraestructura.Persistencia.Loggers
{
    2 referencias
    public class FileLogger : ILoggerStrategy
    {
        private readonly string _ruta;

        1 referencia
        public FileLogger()
        {
            string carpetaEjecucion = AppDomain.CurrentDomain.BaseDirectory;
            _ruta = Path.Combine(carpetaEjecucion, "logs.txt");

            if (!File.Exists(_ruta))
            {
                using (var stream = File.Create(_ruta)) { }
            }
        }

        2 referencias
        public async Task Registrar(Log log)
        {

            string linea = $"{log.FechaHora} [{log.Tipo}] {log.Mensaje}";

            await File.AppendAllTextAsync(_ruta, linea + Environment.NewLine, Encoding.UTF8);
        }
    }
}
```

Database Logger almacena los Logs en la base de datos, es utilizado para los eventos.

```
namespace Infraestructura.Persistencia.Loggers
{
    2 referencias
    public class DatabaseLogger : ILoggerStrategy
    {
        private  IRepositorio<Log> _repo;

        1 referencia
        public DatabaseLogger (IRepositorio<Log> repo)
        {
            _repo = repo;
        }

        2 referencias
        public async Task Registrar(Log log)
        {
            await _repo.Create(log);
            await _repo.SaveChangesAsync();
        }


    }
}
```

- **Repository + Unit of Work:** asegura consistencia y organización en las operaciones de base de datos.

La app fue diseñada para aceptar repositorios base que implementan IRepositorio, permitiendo hacer uso de las operaciones CRUD en cualquier entidad simple creada.

Sin embargo, en varios casos, se utilizo una interfaz+Clase concreta para añadir funcionalidades especificas de las entidades como obtener un dato por otro que solo existe en esa entidad, ect.

```
namespace Dominio.Interfaces
{
    20 referencias
    public interface IRepositorio <T>
    {
        13 referencias
        public Task Create(T Objeto);
        11 referencias
        public Task Update(T Objeto);

        10 referencias
        public Task<T> Get(int id);
        9 referencias
        public Task<List<T>> GetAll();

        8 referencias
        public Task<int >SaveChangesAsync();

    }
}
```

```
C# Usuario.cs
Interfaces
C# IDiaHabilitadoRepo.cs
C# IEmailService.cs
C# IEstacionRepositorio.cs
C# ILoggerStrategy.cs
C# IPlanificacionRepositorio.cs
C# IRepositorio.cs
C# IReservaRepositorio.cs
C# ISlotRepositorio.cs
C# ITokenRepositorio.cs
C# IUnitOfWork.cs
C# IUsuarioRepository.cs
Infraestructura
```

- **Unit of work:** representa una funcionalidad en conjunto u operaciones que necesitan atomicidad y consistencia como lo son las transacciones en gestión de cupos y disponibilidad.

```
using System.Threading.Tasks;

namespace Aplicacion.Interfaces
{

    public interface IUnitOfWork : IDisposable
    {

        IReservaRepositorio ReservaRepo { get; }


         IDiaHabilitadoRepo DiaHabilitadoRepo { get; }

        IEstacionRepositorio EstacionRepo { get; }

        IPlanificacionRepositorio PlanificacionRepo { get; }

        ISlotRepositorio SlotRepo { get; }

        IRepositorio<Turno> TurnoRepo { get; }

        ITokenRepositorio TokenCancelacionRepo { get; }


        IUsuarioRepository UsuarioRepository { get; }

        7 referencias
        Task<int> SaveChangesAsync();

        5 referencias
        Task<IDbContextTransaction> BeginTransactionAsync();
    }

}
```

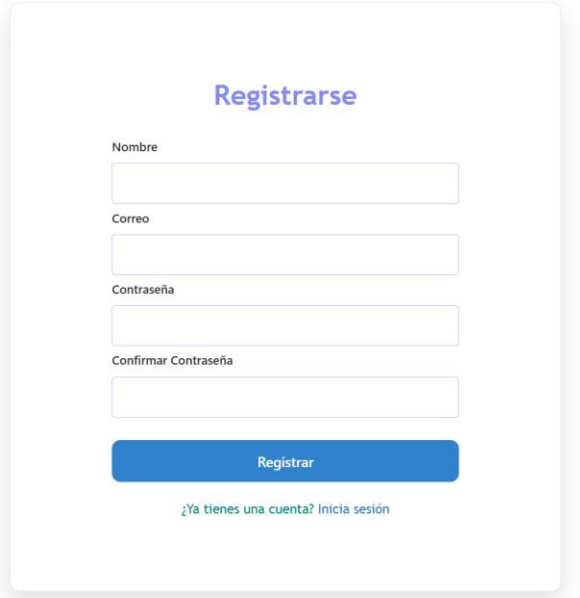
La clase que la implementa es esta. Esta clase es utilizada en los servicios de aplicación como lo son ReservaService, Planificación, ect,. Al tener acceso a varios repositorios y con funciones para transacciones, la entidad permite realizar operaciones haciendo uso de múltiples entidades y repositios de esas entidades, permitiendo aplicar el principio de atomicidiad, consistentencia.

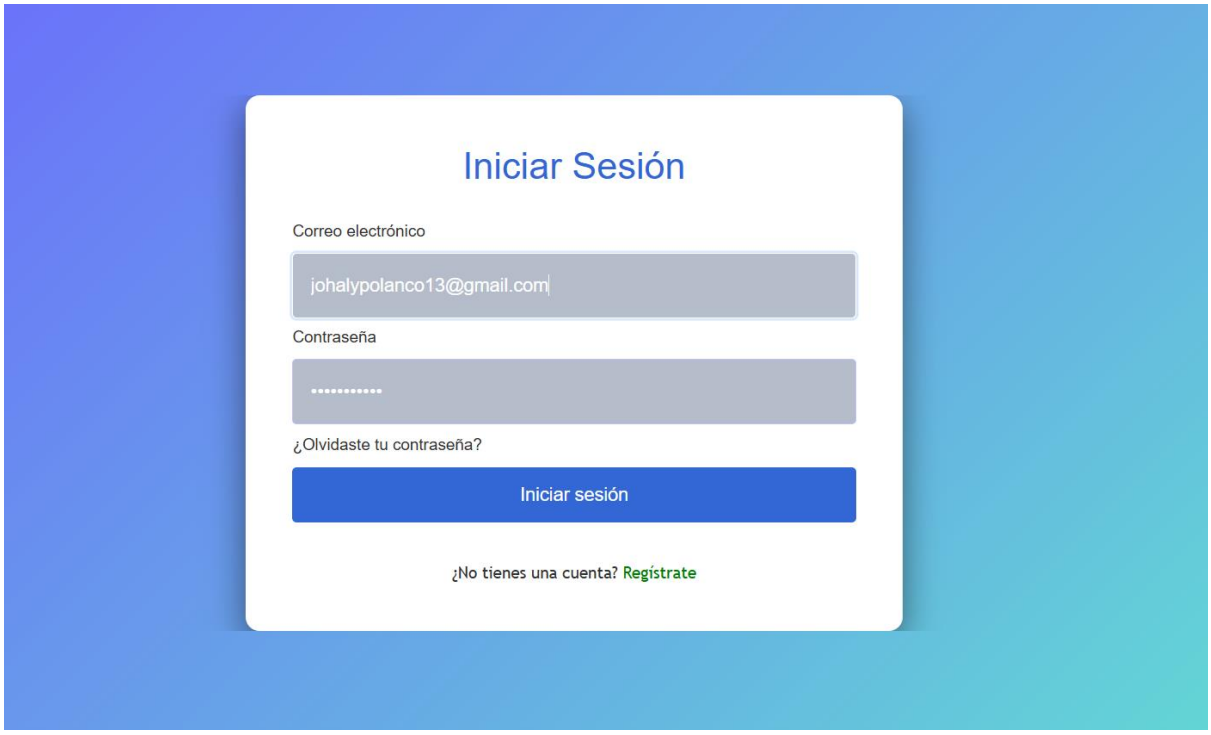
```
using System.Threading.Tasks;

namespace Infraestructura.Persistencia
{
    2 referencias
    public class UnitOfWork : IUnitOfWork
    {
        private readonly SistemaReservasContext _context;

        4 referencias
        public IDiaHabilitadoRepo DiaHabilitadoRepo { get; }
        3 referencias
        public IEstacionRepositorio  EstacionRepo { get; }
        3 referencias
        public IPlanificacionRepositorio PlanificacionRepo { get; }
        6 referencias
        public ISlotRepositorio SlotRepo{ get; }
        1 referencia
        public IRepositorio<Turno> TurnoRepo { get; }

        7 referencias
        public ITokenRepositorio TokenCancelacionRepo { get; }

        5 referencias
        public IUsuarioRepository UsuarioRepository { get; }

        10 referencias
        public IReservaRepositorio ReservaRepo { get; }


        0 referencias
        public UnitOfWork(
            SistemaReservasContext context,
            IDiaHabilitadoRepo diaHabilitadoRepo,
            IEstacionRepositorio  estacionRepo,
           IPlanificacionRepositorio planificacionRepo,
            ISlotRepositorio slotRepo,
            IUsuarioRepository usuarioRepository,
            IRepositorio<Turno> turnoRepo,
            IReservaRepositorio reservaRepositorio,
             ITokenRepositorio tokenCancelacionRepo

            )
        {
            _context = context;
            DiaHabilitadoRepo = diaHabilitadoRepo;
            EstacionRepo = estacionRepo;
            PlanificacionRepo = planificacionRepo;
            SlotRepo = slotRepo;
            UsuarioRepository = usuarioRepository;
            ReservaRepo = reservaRepositorio;
            TokenCancelacionRepo = tokenCancelacionRepo;

        }

        7 referencias
        public async Task<int> SaveChangesAsync() => await _context.SaveChangesAsync();

        0 referencias
        public void Dispose() => _context.Dispose();

        5 referencias
        public async Task<IDbContextTransaction> BeginTransactionAsync()
        {
            return await _context.Database.BeginTransactionAsync();
        }
    }

}
```

Tecnologías utilizadas

1. **Frontend:** React.js (componentes, calendarios y funcionalidades nativas).
2. **Backend:** ASP.NET Core.
3. **ORM:** Entity Framework.
4. **Correo:** MailKit.
5. **Patrones de diseño:** Singleton, Adapter, Repository, Unit of Work, Strategy.

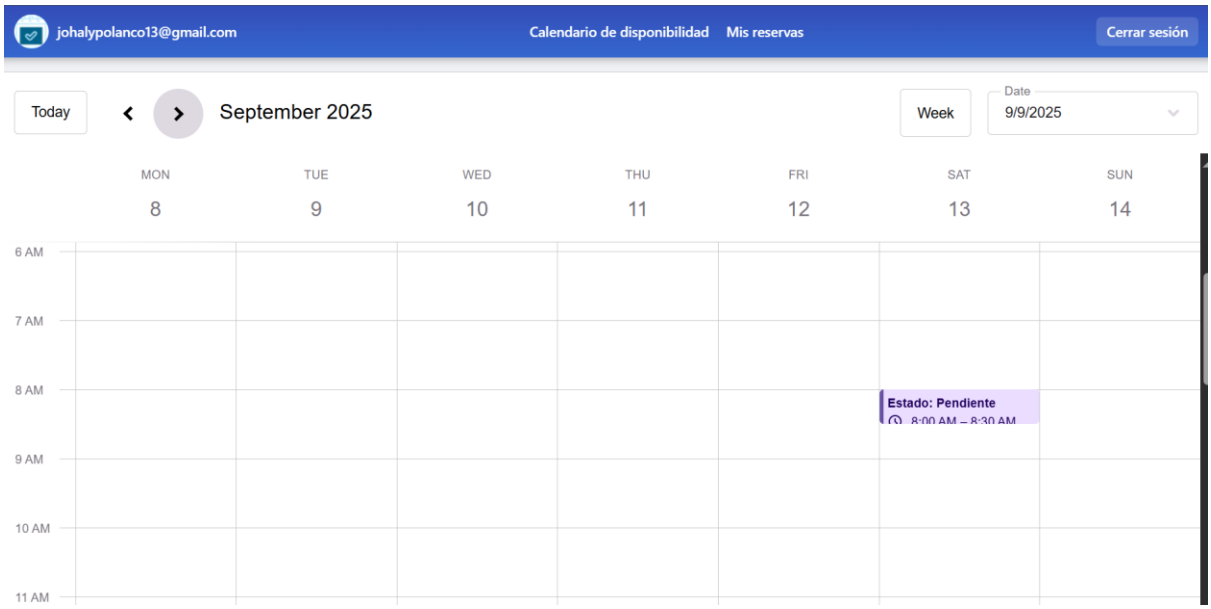
Capturas

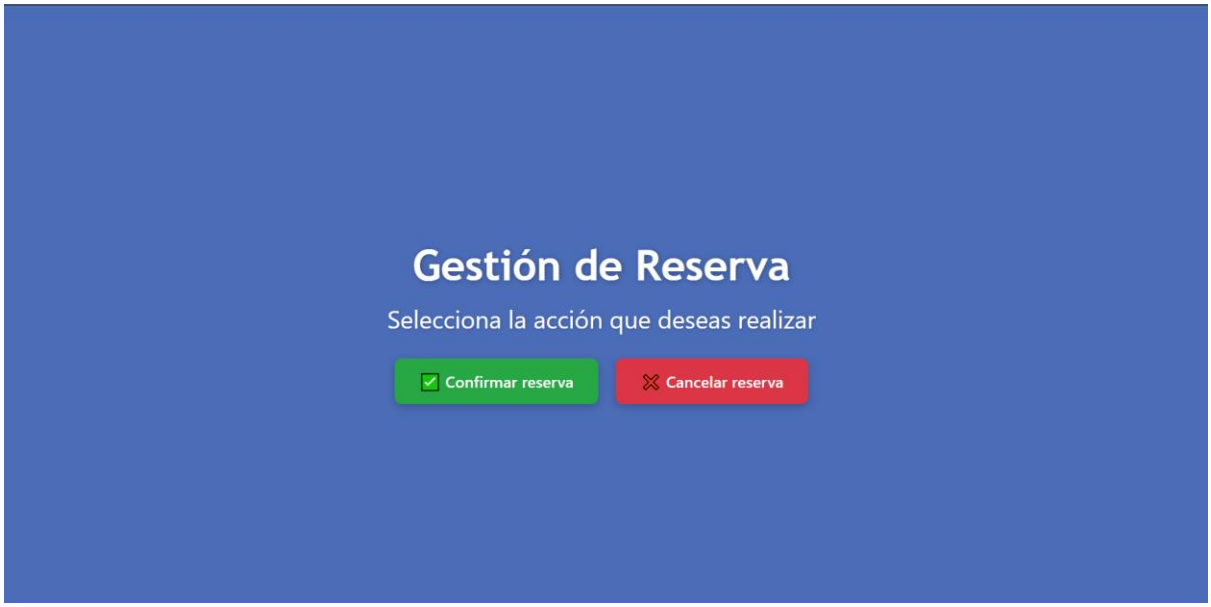




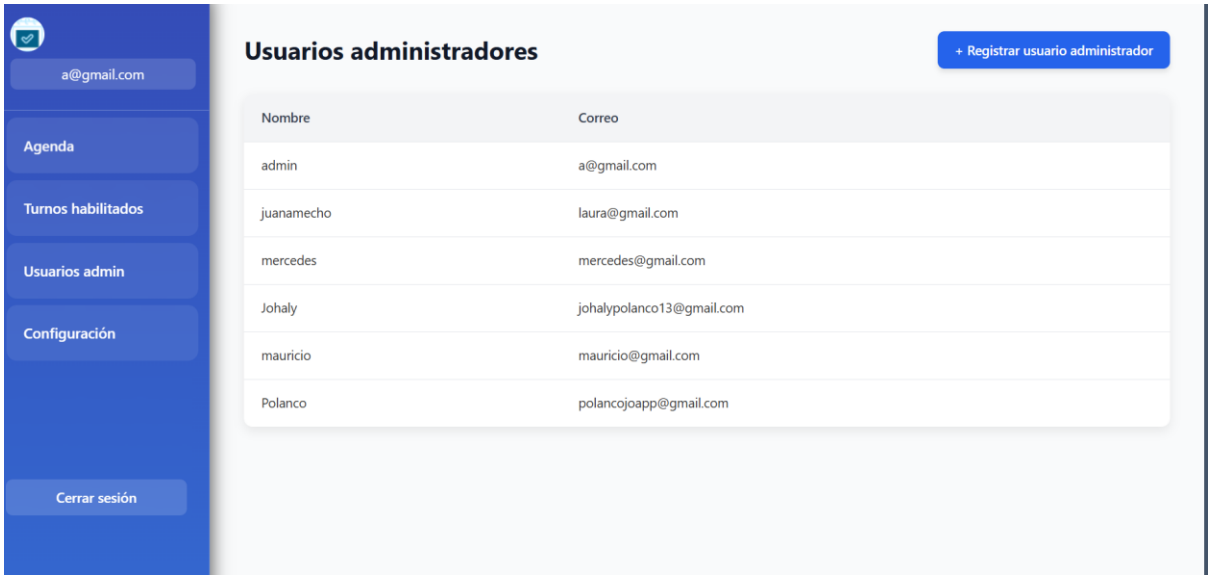
Vista de usuario

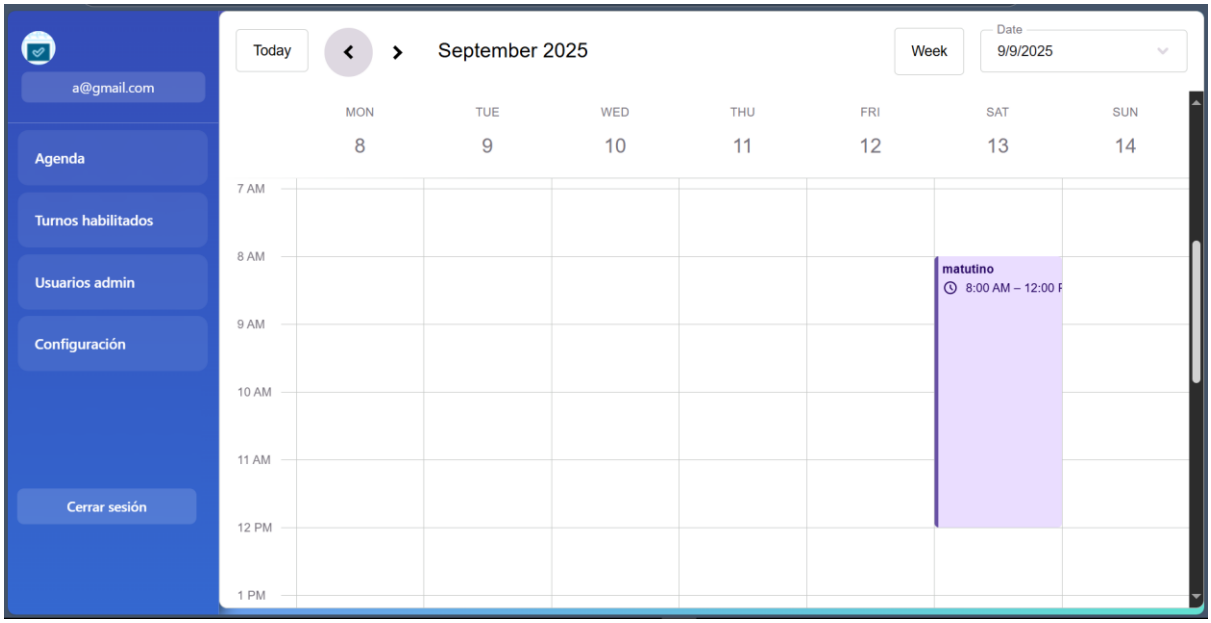
johalypolanco13@gmail.com
Calendario de disponibilidad
Mis reservas
Cerrar sesión
Selecciona un slot disponible (verde) para reservar tu turno
Hoy
8 - 14 sept 2025
Mes
Semana
Día
lun 8/9
mar 9/9
mié 10/9
jue 11/9
vie 12/9
sáb 13/9
dom 14/9
Todo el día
8
9
10
11
08:00:00 - 08:30:00
08:30:00 - 09:00:00
09:00:00 - 09:30:00
09:30:00 - 10:00:00
10:00:00 - 10:30:00
10:30:00 - 11:00:00
11:00:00 - 11:30:00

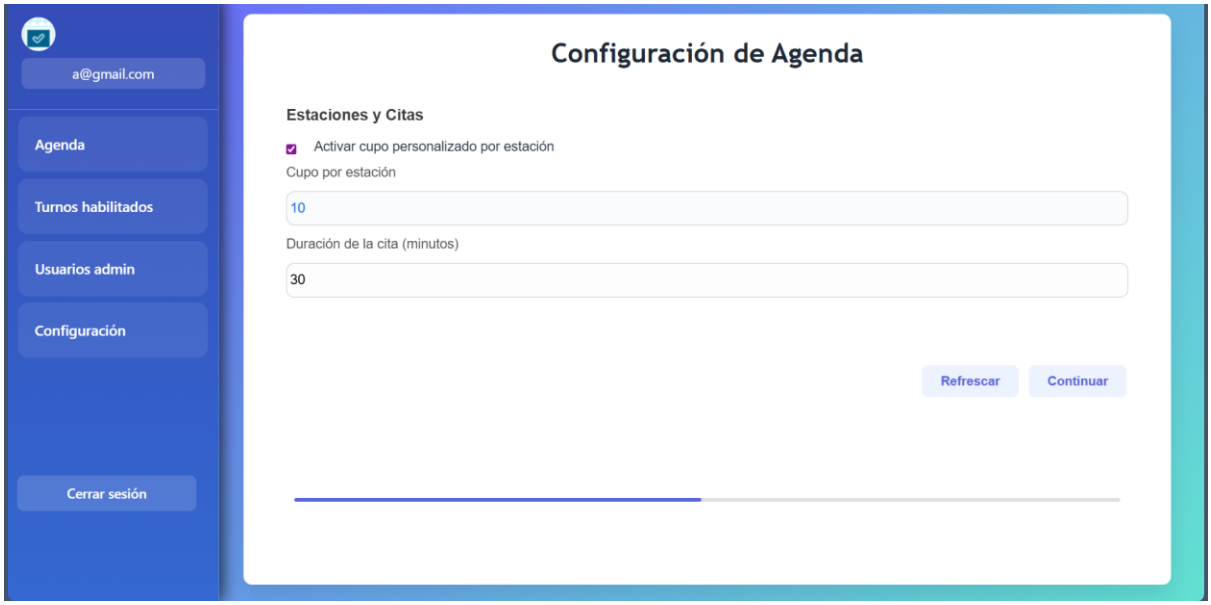

johalypolanco13@gmail.com
Calendario de disponibilidad
Mis reservas
Cerrar sesión
Today
September 2025
Week
Date
9/9/2025
MON 8
TUE 9
WED 10
THU 11
FRI 12
SAT 13
SUN 14
6 AM
7 AM
8 AM
9 AM
10 AM
11 AM
Estado: Pendiente

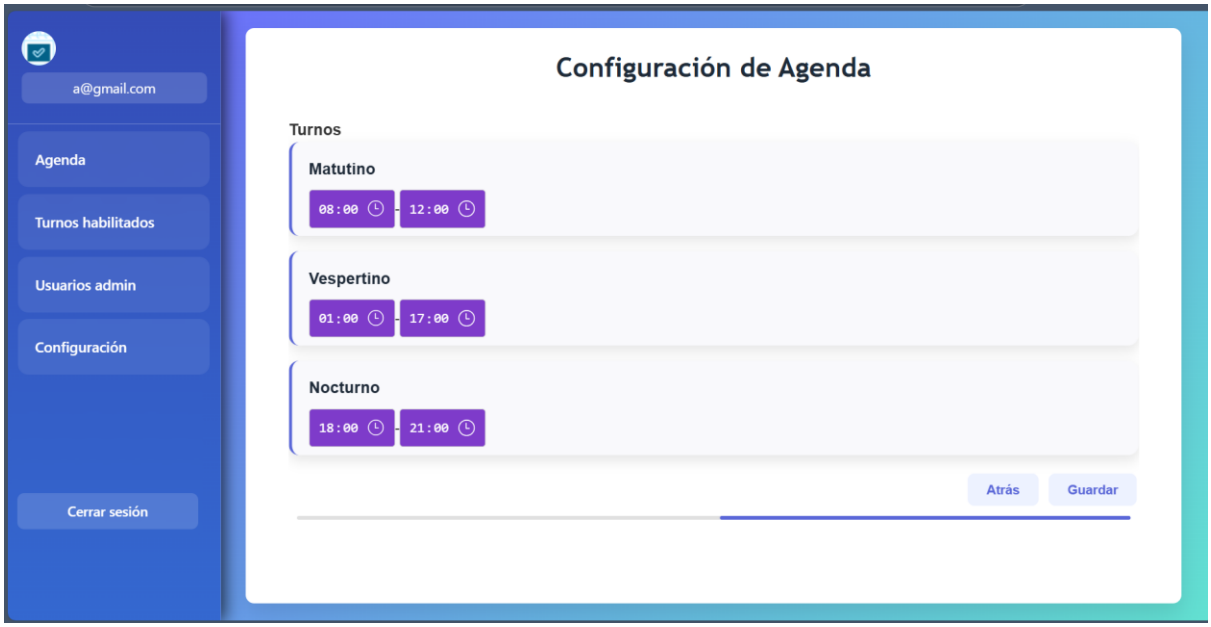


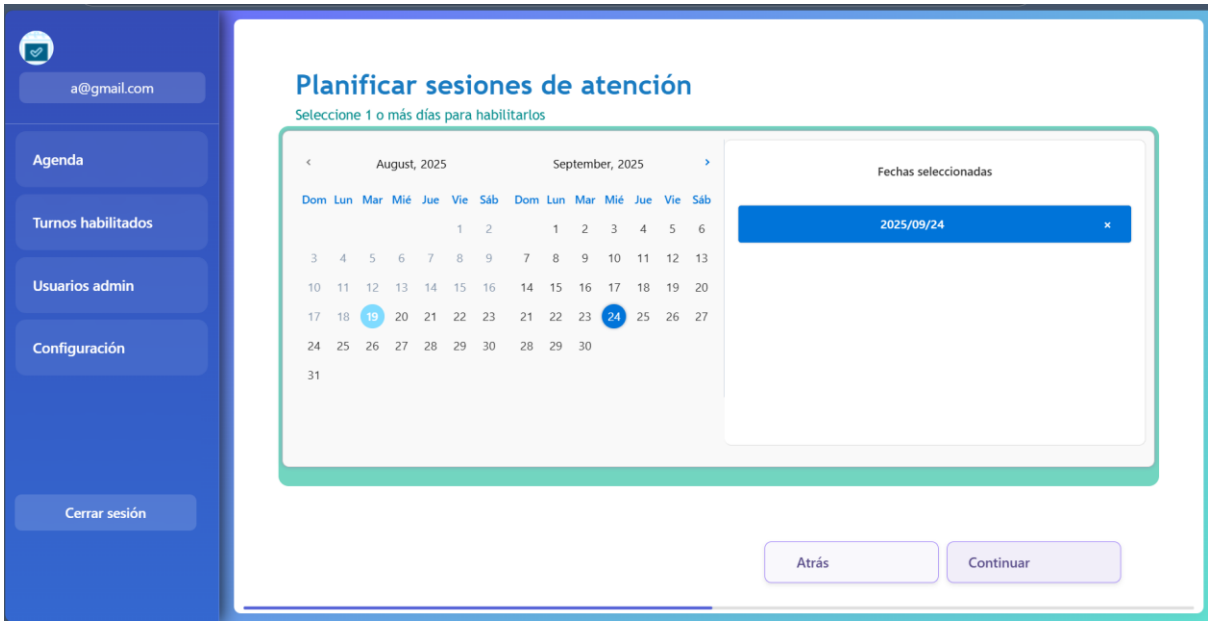
Vista de admin

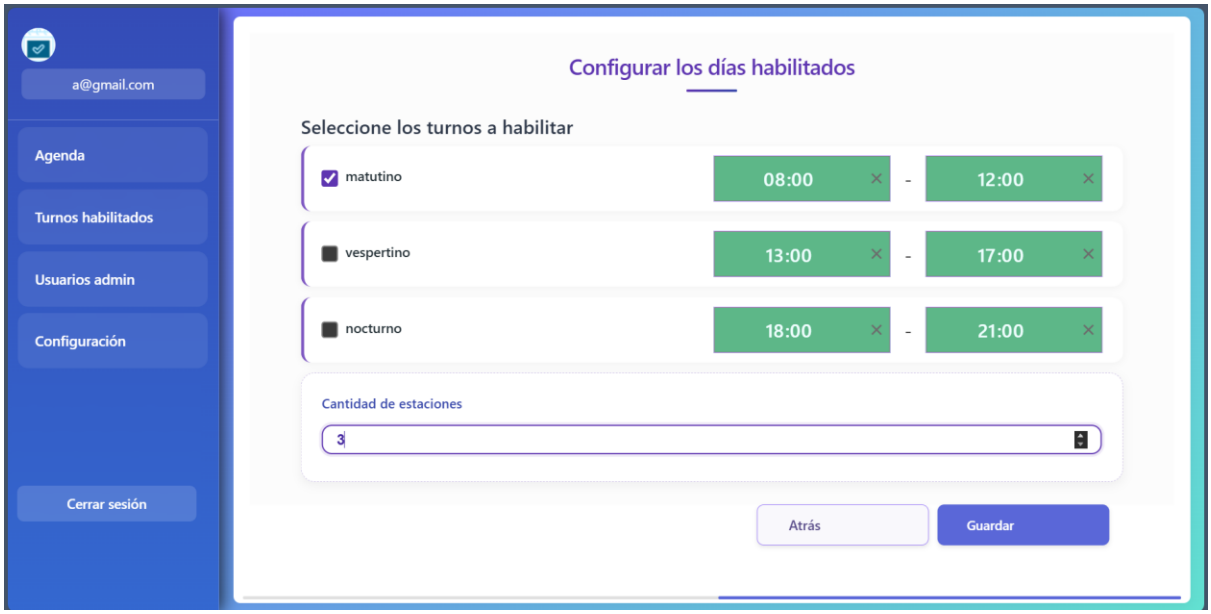



a@gmail.com
Agenda
Turnos habilitados
Usuarios admin
Configuración
Cerrar sesión
Today
September 2025
Week
Date
9/9/2025
MON 8
TUE 9
WED 10
THU 11
FRI 12
SAT 13
SUN 14
7 AM
8 AM
9 AM
10 AM
11 AM
12 PM
1 PM
matutino
8:00 AM – 12:00 P


a@gmail.com
Agenda
Turnos habilitados
Usuarios admin
Configuración
Cerrar sesión
Configuración de Agenda
Estaciones y Citas
Activar cupo personalizado por estación
Cupo por estación
10
Duración de la cita (minutos)
30
Refrescar
Continuar


a@gmail.com
Agenda
Turnos habilitados
Usuarios admin
Configuración
Cerrar sesión
Configuración de Agenda
Turnos
Matutino
08:00
12:00
Vespertino
01:00
17:00
Nocturno
18:00
21:00
Atrás
Guardar


a@gmail.com
Agenda
Turnos habilitados
Usuarios admin
Configuración
Cerrar sesión
Planificar sesiones de atención
Seleccione 1 o más días para habilitarlos
August, 2025
September, 2025
Dom Lun Mar Mié Jue Vie Sáb
Fechas seleccionadas
2025/09/24
Atrás
Continuar


a@gmail.com
Agenda
Turnos habilitados
Usuarios admin
Configuración
Cerrar sesión
Configurar los días habilitados
Seleccione los turnos a habilitar
matutino
08:00
12:00
vespertino
13:00
17:00
nocturno
18:00
21:00
Cantidad de estaciones
3
Atrás
Guardar